COMP2040: Operating Systems
Lab 01: C Foundations, OS Services and System Calls

Daniela Méndez Ramírez
College of Engineering and Computer Science
VinUniversity, Hanoi, Vietnam
Student ID: VU2614027

***Abstract*—This lab is about the basic functions of an operating system, studied through the C language. During the lab, several topics are covered: how to print text on the screen, how to write modular code by splitting functions into several files, how to use system calls to understand input and output (I/O), and how to check the size and permissions of a file. This report shows the commands used, the results obtained, and the answers to the lab questions.**

***Index Terms*—Operating Systems, System Calls, POSIX, File Descriptors, File Permissions, C Programming.**

I. Introduction

An operating system offers services to programs. Programs ask for these services using system calls, and usually they do it through library functions such as `printf()`. This lab explains how these functions work and how they connect with the Linux kernel.

The lab has five tasks: compiling and running a program, using header files and several source files, reading and writing files with system calls, checking the size of a file, and checking the permissions of a file. For each task, this report includes the commands, the output, and the answers to the questions.

All the code and files are available in this public repository:

https://github.com/Danym2407/Operative-Systems---Lab

II. Task 1: Edit, Compile, Run

In this task, a basic C program is compiled and executed. The program is modified to print the name of the author, and the compilation process is explained.

A. Commands and Observed Output

The file `task1_hello.c` was compiled and executed with the commands below. The terminal output is shown in Fig. 1 (Appendix).

```
$ gcc -Wall -Wextra task1_hello.c -o task1_hello
$ ./task1_hello
Hello World, we are in Operating Systems Lab Class
My Name is Daniela
```

B. Analysis and Questions

- **(b) Compilation output:**
  The compiler created an executable file named `task1_hello`. This name was chosen with the option `-o task1_hello`. Without this option, the file would be named `a.out`.
- **(c) `printf()` vs. `write()`:**
  The manual pages [1], [4] show the difference between these two functions:
  - `printf(3)` is in section 3 of the manual (*Library Functions*). It runs in user space and its job is to format text, for example with specifiers like `%d`. It cannot access the screen or other hardware directly.
  - `write(2)` is in section 2 of the manual (*System Calls*). It asks the kernel to take bytes from a buffer and send them to a file descriptor, for example `fd = 1`, which is the standard output (the terminal).

  In summary, `printf()` prepares the text in memory, and then it calls `write()` so the kernel can show the text on the screen.

III. Task 2: Header Files and Multiple Source Files

In this task, a program is divided into several files. A header file shows what the functions do, and a source file contains the code of the functions. This is called modular programming.

A. Commands and Observed Output

The files `main.c` and `math_operations.c` were compiled together with the flags `-Wall` and `-Wextra`. The result is the executable `math_demo`. The terminal output is shown in Fig. 2 (Appendix).

```
$ gcc -Wall -Wextra main.c math_operations.c -o
    math_demo
$ ./math_demo
10 + 5 = 15
10 - 5 = 5
10 * 5 = 50
10 / 5 = 2.00
```

B. Analysis and Questions

- **(b) Header file vs. source file:**

  - The header file (`math_operations.h`) contains the function declarations (prototypes). A declaration shows the name of the function, its return type, and its parameters. The header also has include guards to avoid including it twice.
  - The source file (`math_operations.c`) contains the function definitions. This is the real code of each function. For example, the division function checks if the divisor is zero before dividing.
- **(c) Why `main.c` includes `math_operations.h`:** The compiler translates each `.c` file separately. When it compiles `main.c`, it does not know the math functions yet. The header file tells the compiler their names, return types, and parameters. In this way, the compiler can check that the function calls are correct. After that, the linker joins the compiled files into one executable.

IV. Task 3: File I/O with System Calls

In this task, a file is created, written, and read using four system calls: `open()`, `write()`, `read()`, and `close()` [2]–[5].

A. Background

A **system call** is a request from a program to the operating system. A normal program cannot use the disk directly. Only the *kernel*, the main part of the operating system, can do that. So the program asks the kernel, and the kernel does the work.

After a file is opened, the kernel gives the program a number called a **file descriptor**. The program uses this number to talk about the file. Every program already has three descriptors: 0 is the keyboard (input), 1 is the screen (output), and 2 is the screen for errors. For this reason, the first file that a program opens usually gets the number 3.

B. Program Steps

The program follows six steps:

1) `open()` creates the file `lab01.txt` for writing.
2) `write()` puts the message in the file (39 bytes).
3) `close()` closes the file.
4) `open()` opens the same file again, only for reading.
5) `read()` copies the text from the file into memory.
6) `close()` closes the file, and `printf()` shows the text.

After each system call, the program checks if the result is `-1`. This means there was an error, and `perror()` prints the reason.

C. Commands and Observed Output

The program was compiled and executed. It wrote 39 bytes and then read the same 39 bytes. Then, `cat` showed the content of the file, and `ls -l` showed its details. The terminal output is shown in Fig. 3 (Appendix).

```
$ gcc -Wall -Wextra task3.c -o task3
$ ./task3
Wrote 39 bytes to lab01.txt
Read 39 bytes from lab01.txt:
Hello from a COMP2040 system call lab!

$ cat lab01.txt
Hello from a COMP2040 system call lab!

$ ls -l lab01.txt
-rwxrwxrwx 1 root root 39 Oct  4 08:54 lab01.txt
```

Both numbers are 39, so all the data was written and read correctly. The message has 38 characters plus the newline (`\n`), which makes 39 bytes.

D. Analysis and Questions

- **File descriptor (`int fd`):** It is a small number that identifies an open file. The program uses it in `write()`, `read()`, and `close()` to say which file to use. If `open()` fails, it returns `-1` and saves the reason in the variable `errno`.
- **Flags of `open()`:** Flags tell the kernel how to open the file. Three flags were joined with the symbol `|`:
  - `O_WRONLY`: open the file for writing only.
  - `O_CREAT`: create the file if it does not exist.
  - `O_TRUNC`: if the file exists, delete the old content.

  The second `open()` uses `O_RDONLY`, which means reading only.
- **Return values of `write()` and `read()`:** Both return how many bytes they moved, or `-1` if there was an error. The program prints these numbers to confirm the result.
- **Why `close()` is needed:** A program can have only a limited number of open files. The function `close()` frees the file descriptor so it can be used again.
- **Requested permissions vs. observed permissions:** The last argument of `open()` is the permission mode, here `0644`. The three digits are for the owner, the group, and the others. The number 6 means read (4) plus write (2). The number 4 means read only. So `0644` is `-rw-r--r--`. However, `ls -l` shows `-rwxrwxrwx`. This happens because the file is in a Windows folder used by WSL. Windows does not save Linux permissions in the same way, so WSL shows full permissions. In a normal Linux folder, the result would be `-rw-r--r--`.
- **Null terminator in `read()`:** In C, a string must end with the null character (`'\0'`). The function `read()` does not add it. Without it, `printf()` could print strange characters after the text.
  For this reason, the program reads at most `sizeof(buffer) - 1` bytes to leave one free space. Then, it writes `'\0'` in `buffer[bytes_read]`. Now the buffer is a valid string.

V. Task 4: File Size Using `stat()`

In this task, the system call `stat()` [6] is used to get the size of a file.

A. Background

Every file has **metadata**. Metadata is information about the file, not the text inside it. For example, the size of the file, its permissions, and its owner are metadata.

The system call `stat()` asks the kernel for this information. The program does not need to open or read the file. The kernel saves the answer in a structure called `struct stat`. This structure has many fields. One of them is `st_size`, which is the size of the file in bytes.

A **byte** is a small unit of data. In a text file with simple English letters, each character uses 1 byte.

B. Commands and Observed Output

The program was compiled and executed. First, it was run without arguments to check the usage message. Then, it was run with `lab01.txt` before and after adding text to the file. The terminal output is shown in Fig. 4 (Appendix).

```
$ gcc -Wall -Wextra task4.c -o task4
$ ./task4
Usage: ./task4 <file>

# File size before the edit:
$ ./task4 lab01.txt
File: lab01.txt
Size: 39 bytes

# The text "Operating Systems Lab." (22 characters)
     was added to lab01.txt
# File size after the edit:
$ ./task4 lab01.txt
File: lab01.txt
Size: 61 bytes
```

When the program runs without a file name, it prints a usage message. This tells the user how to run the program correctly.

C. Analysis and Questions

- **(a) File size before and after:**
  Before the edit, the size was **39 bytes**. This is the text written in Task 3. After the edit, the size was **61 bytes**. The file became 22 bytes bigger.
- **(b) Field for the file size:**
  The field is `st_size`, inside `struct stat`. Its type is `off_t`, and it is defined in `<sys/stat.h>` [6].
- **(c) Why the size changed:**
  Each character in the file uses 1 byte. The new text has 22 characters, so the file grew by 22 bytes: 39 + 22 = 61. The field `st_size` always shows the total number of bytes in the file. If the file changes, this number changes too.

VI. Task 5: File Permissions Using `stat()`

In this task, the system call `stat()` [6] is used to read the permissions of a file.

A. Background

In Linux, every file has **permissions**. They say who can do what with the file. There are three actions:

- **r** (read): see the content of the file.
- **w** (write): change the content of the file.
- **x** (execute): run the file as a program.

There are also three types of users: the **owner** (the person who owns the file), the **group** (a set of users), and the **others** (everyone else). Each type has its own `r`, `w`, and `x`, so there are nine permissions in total. For example, `rw-r--r--` means that the owner can read and write, and the group and the others can only read.

The kernel saves all nine permissions as bits inside the field `st_mode` of `struct stat`. A **bit** is a value of 0 or 1. If the bit is 1, the permission is allowed. If the bit is 0, it is not allowed.

To check one bit, the program uses a **mask** and the bitwise AND operator (`&`). A mask is a number that has only the bit we want to check. If `mode & mask` is not zero, the bit is set.

B. Commands and Observed Output

The program was compiled and executed on `lab01.txt`. Then, the result was compared with the output of `ls -l`. The terminal output is shown in Fig. 5 (Appendix).

```
$ gcc -Wall -Wextra task5.c -o task5
$ ./task5 lab01.txt
File: lab01.txt
Permissions: rwxrwxrwx

$ ls -l lab01.txt
-rwxrwxrwx 1 root root 39 Oct  4 08:54 lab01.txt
```

Both outputs show the same permissions. In `ls -l`, the first character is the file type (`-` means a normal file). The next nine characters are the permissions.

C. Analysis and Questions

- **How the permissions are read:**
  The program checks each of the nine bits with a mask:
  - Owner: `S_IRUSR` (read), `S_IWUSR` (write), `S_IXUSR` (execute).
  - Group: `S_IRGRP` (read), `S_IWGRP` (write), `S_IXGRP` (execute).
  - Others: `S_IROTH` (read), `S_IWOTH` (write), `S_IXOTH` (execute).

  The names are easy to read: `S_I` is a prefix, then `R`, `W`, or `X` is the action, and `USR`, `GRP`, or `OTH` is the type of user.
- **Helper function `permission_char()`:**
  This function checks one bit with `mode & mask`. If the bit is set, it returns `'r'`, `'w'`, or `'x'`. If not, it returns a hyphen (`'-'`). The program calls it nine times and prints the nine characters together.
- **Comparison with `ls -l`:**
  The result of the program (`rwxrwxrwx`) is the same as the permissions shown by `ls -l`. This confirms that the program reads and decodes the permissions correctly.
- **Why all permissions are allowed:**
  The file has full permissions (`rwxrwxrwx`) because it is in a Windows folder used by WSL. Windows does not

save Linux permissions in the same way, so WSL shows full permissions for every file. In a normal Linux folder, a new file would show `rw-r--r--`, like in Task 3.

AI Tool Disclosure

The AI assistant Claude (Anthropic) [7] was used in this lab for three purposes: to improve the English writing of this report, to explain basic concepts, and to help set up the `gcc` compiler in the Linux terminal (WSL). The lab code, the experiments, and the results are the work of the author. The full conversation is available at: https://claude.ai/share/7e3e1a7a-0966-4cc4-a672-d3a7facdf45b

References

[1] Linux man-pages project, "printf(3) – Linux manual page." [Online]. Available: https://man7.org/linux/man-pages/man3/printf.3.html

[2] Linux man-pages project, "open(2) – Linux manual page." [Online]. Available: https://man7.org/linux/man-pages/man2/open.2.html

[3] Linux man-pages project, "read(2) – Linux manual page." [Online]. Available: https://man7.org/linux/man-pages/man2/read.2.html

[4] Linux man-pages project, "write(2) – Linux manual page." [Online]. Available: https://man7.org/linux/man-pages/man2/write.2.html

[5] Linux man-pages project, "close(2) – Linux manual page." [Online]. Available: https://man7.org/linux/man-pages/man2/close.2.html

[6] Linux man-pages project, "stat(2) – Linux manual page." [Online]. Available: https://man7.org/linux/man-pages/man2/stat.2.html

[7] Anthropic, "Claude," large language model, conversation used for this report. Accessed: Oct. 4, 2026. [Online]. Available: https://claude.ai/share/7e3e1a7a-0966-4cc4-a672-d3a7facdf45b

APPENDIX

This appendix compiles the screenshots documenting the compilation steps and execution outputs across all laboratory tasks.

A. Task 1: Basic Compilation and System Calls

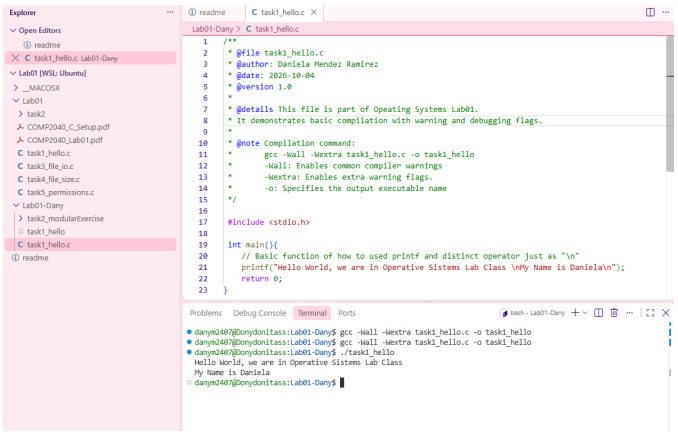


Figure 1. Compilation flags and execution output for Task 1.

B. Task 2: Modular Programming and Linking

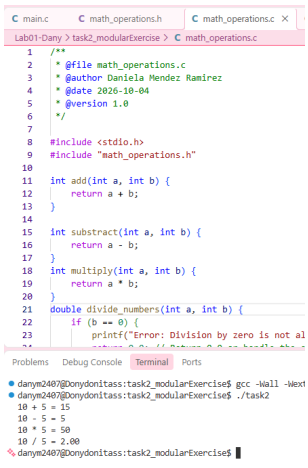


Figure 2. Multi-file compilation and arithmetic results for Task 2.

C. Task 3: File I/O via POSIX System Calls

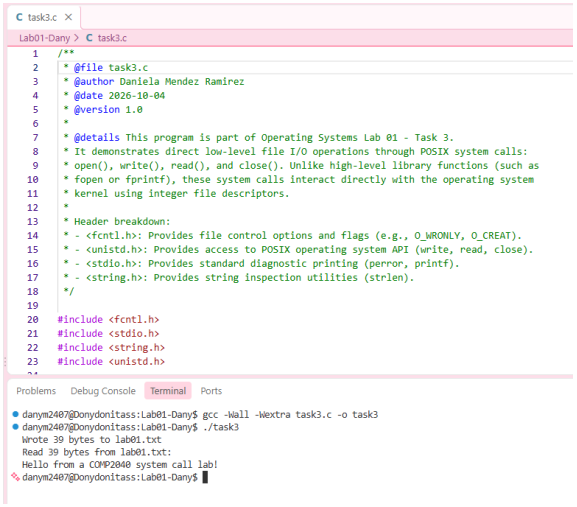


Figure 3. Terminal log of `open()`, `write()`, and `read()` operations.

D. Task 4: Metadata and File Size Inspection



Figure 4. File size before and after modification using `stat()`.

E. Task 5: File Permission Decoding

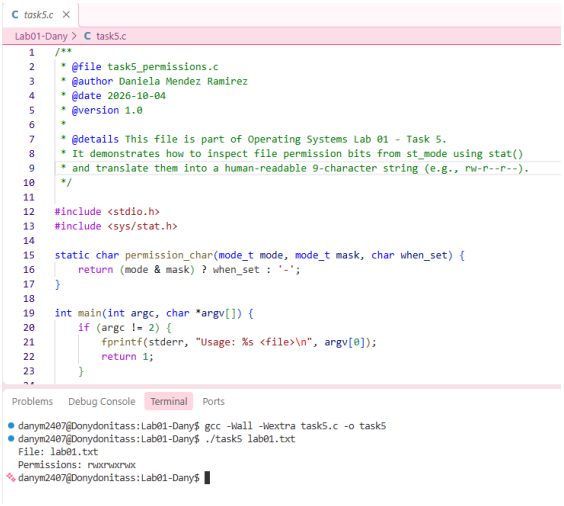


Figure 5. Permission bits from `st_mode` compared against `ls -l`.